
Dreaming Smoothly and Sample Efficiently with Gradient Penalized Latent Dynamics

Romil V. Sonigra

Department of Electrical and Computer Engineering
Texas A&M University
College Station, TX 77843
romils@tamu.edu

P. R. Kumar

Department of Electrical and Computer Engineering
Texas A&M University
College Station, TX 77843
prk@tamu.edu

Abstract

Model-based reinforcement learning improves sample efficiency by learning a world model. However, existing latent world models such as DreamerV3 do not explicitly enforce local smoothness in their learned transition dynamics, leaving a useful inductive bias for transition dynamics learning unexploited. We propose GPLD, a gradient-penalized latent dynamics regularizer for DreamerV3 that applies a row-wise Jacobian penalty to the posterior latent distribution to encourage locally smooth transition learning. We show that this penalty can be interpreted as the continuous-latent analog of finite-difference smoothing of transition laws in discrete embedded-state MDPs, and estimate it efficiently using Hutchinson-style stochastic probes. Empirically, across DeepMind Control proprioceptive tasks, GPLD improves aggregate sample efficiency, with particularly strong gains on higher-complexity locomotion environments. On more challenging quadruped tasks, GPLD reaches high-return behavior earlier and exhibits more consistent late-stage learning over longer horizons. Explicit local smoothness regularization is a simple and effective way to improve latent world models for smooth continuous control environments. Code for GPLD is available at github.com/romils9/gpld-mbrl.

1 Introduction

Model-based reinforcement learning (MBRL) improves sample efficiency by learning a predictive world model that can be used for planning and policy optimization (Ha and Schmidhuber, 2018). DreamerV3 has demonstrated that latent world models can scale to diverse continuous-control tasks by learning compact recurrent state-space representations and training policies through imagined rollouts (Hafner et al., 2025). Despite this success, standard latent world models do not explicitly exploit a basic structural prior of many continuous-control systems: nearby states induce similar short-horizon transition behavior. Encoding this prior directly in the learned latent dynamics can help the world model share information across nearby states, reducing the amount of interaction data needed to learn useful transition structure.

We study how to exploit this local smoothness prior through *gradient-penalized latent dynamics* (GPLD), a differentiable regularizer for latent world-model dynamics. This regularizer is motivated through a discrete-to-continuous smoothness argument. In a finite embedded-state MDP, local

smoothness can be imposed by penalizing differences between transition laws at neighboring states. When the state representation is continuous and the transition model is differentiable, these normalized finite differences become directional derivatives of the learned dynamics. Averaging over local directions yields a Frobenius Jacobian penalty. This provides a principled route from neighborhood smoothing in tabular MDPs to the gradient penalty used by GPLD.

We implement GPLD in DreamerV3 and evaluate it on DeepMind Control tasks. Empirically, GPLD improves aggregate sample efficiency on proprioceptive tasks, with the largest gains attained on higher-complexity locomotion environments. On difficult quadruped tasks, the benefit becomes clearer over longer horizons, where GPLD reaches high-return behavior earlier and exhibits more consistent late-stage learning. Pixel-observation results show milder aggregate gains, suggesting that the effect of latent smoothness regularization is weaker when dynamics learning is coupled to high-dimensional visual encoding.

Our contributions are:

1. A discrete-to-continuous justification showing how finite-difference smoothing of tabular transition laws leads to Frobenius Jacobian regularization in differentiable latent models.
2. GPLD, a posterior Jacobian regularizer for DreamerV3 that explicitly encourages local smoothness in latent probability maps, yielding a normalized aggregate gain of 17.7% across the DeepMind Control proprioceptive benchmark.
3. Evaluation of GPLD on DeepMind Control, showing a 34.6% normalized aggregate gain on higher-complexity proprioceptive locomotion tasks, clearer long-horizon benefits on difficult quadruped tasks, and ablations over the main design choices.

Code for GPLD is available at github.com/romils9/gpld-mbrl.

2 Related Work

Smoothness in reinforcement learning. Smoothness has long been recognized as important for generalization, stable prediction, and control in reinforcement learning (Boyan and Moore, 1994; Munos and Szepesvári, 2008; Asadi et al., 2018; Christmann et al., 2024). Unconstrained neural networks can produce high-frequency oscillations that degrade stability and generalization (Boyan and Moore, 1994; Rosca et al., 2020; Christmann et al., 2024), and such errors can compound in multi-step value or dynamics prediction (Thrun and Schwartz, 1993; Venkatraman et al., 2015). In model-based RL, Asadi et al. (2018) show that Lipschitz constraints on learned transition models yield tighter multi-step estimation error bounds, directly linking transition smoothness to world-model accuracy. These works motivate smoothness as a useful inductive bias for learned dynamics, but do not provide a local Jacobian regularizer for stochastic latent world models.

Gradient penalties and control smoothness. Gradient penalties provide a differentiable way to encourage local smoothness and have been widely used in representation and generative modeling (Gulrajani et al., 2017; Terjék, 2020; Goodfellow et al., 2015). In reinforcement learning, related methods often regularize value functions, policies, or action trajectories. For example, Wang et al. (2025) apply gradient penalties to Q-functions in model-free RL, while Chen et al. (2024) introduce Lipschitz-constrained policies as a differentiable alternative to task-specific reward-smoothing heuristics. Other control approaches encourage smooth behavior through penalties on mechanical energy or low-pass filtering (Fu et al., 2021; Peng et al., 2020). In contrast, GPLD targets the learned world model itself: it applies a row-wise Jacobian penalty to the posterior latent probability map in DreamerV3, improving the latent dynamics used for inference and imagination.

Global versus local smoothness constraints. Global smoothness methods such as spectral normalization constrain the Lipschitz constant of neural networks by bounding layer-wise spectral norms (Gogianu et al., 2021; Bjorck et al., 2022). However, global constraints can be overly restrictive because the global Lipschitz constant may arise from rare, unseen, or irrelevant regions of the input space (Dherin et al., 2022). GPLD instead imposes a local, data-dependent smoothness penalty on sampled latent inputs during world-model training. This distinguishes GPLD from global normalization methods and from policy-smoothing approaches: GPLD regularizes the posterior

transition representation used by a model-based agent, while leaving the rest of the world model unconstrained except through the training objective.

3 A discrete-to-continuous justification for GPLD

We justify GPLD by connecting a discrete local-smoothness prior on transition laws to the continuous Jacobian penalty used in our latent world-model setting. We begin with a finite embedded-state MDP, where smoothness can be imposed by penalizing finite differences of neighboring transition distributions. We then consider a differentiable transition model over continuous state representations and show that these finite differences become directional Jacobian norms, whose isotropic average yields the Frobenius Jacobian norm. This produces the row-wise posterior regularizer used in GPLD.

3.1 Discrete local smoothness prior

We begin with a finite state and finite action MDP. Let $\mathcal{S}$ denote the state space, $\mathcal{A}$ the action space, and let $e : \mathcal{S} \rightarrow \mathbb{R}^d$ be an embedding of states into a metric space. For each action $a \in \mathcal{A}$, let $P(\cdot | s, a) \in \Delta^{|\mathcal{S}|-1}$ denote the transition probability vector, and define the ε -neighborhood of s by

$$\mathcal{N}_\varepsilon(s) := \{k \in \mathcal{S} : \|e(k) - e(s)\|_2 \leq \varepsilon\}.$$

Given a dataset of transitions $\mathcal{D} = \{(s, a, s')\}_{t=1}^T$, let $N(s, a, s')$ denote the number of observed transitions from (s, a) to s' . The standard maximum-likelihood estimator minimizes the negative log-likelihood

$$\mathcal{L}_{\text{MLE}}(P; \mathcal{D}) = - \sum_{s, a, s'} N(s, a, s') \log P(s' | s, a).$$

A natural structural prior is that nearby states should induce similar transition laws. We encode this by coupling neighboring transition vectors through a finite-difference regularizer:

$$\mathcal{L}_{\text{FD}}(P) = \sum_a \sum_s \sum_{k \in \mathcal{N}_\varepsilon(s)} \sum_{s'} \left(\frac{P(s' | k, a) - P(s' | s, a)}{\|e(k) - e(s)\|_2} \right)^2. \quad (1)$$

This yields the regularized objective

$$\min_P \mathcal{L}_{\text{MLE}}(P; \mathcal{D}) + \lambda \mathcal{L}_{\text{FD}}(P).$$

This is the discrete precursor of GPLD: it penalizes normalized local differences of transition laws across neighboring states and therefore explicitly favors locally smooth transition estimates.

3.2 Continuous-state limit

The discrete finite-difference regularizer in (1) leads to a Jacobian penalty when the transition law is represented by a differentiable function over continuous state embeddings. Consider a neighboring state $k \in \mathcal{N}_\varepsilon(s)$. In the embedding space, its displacement from s can be written as

$$e(k) - e(s) = hu, \quad (2)$$

where $h = \|e(k) - e(s)\|_2$ and $u \in \mathbb{R}^d$ is a unit vector. Substituting (2) into a summand of (1) yields

$$\left\| \frac{P(\cdot | k, a) - P(\cdot | s, a)}{\|e(k) - e(s)\|_2} \right\|_2^2 \rightsquigarrow \left\| \frac{f_\theta(x + hu, a) - f_\theta(x, a)}{h} \right\|_2^2,$$

where $x = e(s)$, and f_θ denotes a differentiable continuous-state transition model. Thus, the discrete regularizer naturally gives rise to normalized local finite differences of f_θ .

The small-neighborhood limit $h \rightarrow 0$ then converts these finite differences into directional derivatives.

Let $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^m$ be locally Fréchet differentiable at $x \in \mathbb{R}^d$. Then for any unit vector $u \in \mathbb{R}^d$,

$$\lim_{h \rightarrow 0} \left\| \frac{f_\theta(x + hu) - f_\theta(x)}{h} \right\|_2^2 = \|J_{f_\theta}(x)u\|_2^2. \quad (3)$$

This shows that each normalized finite-difference term converges to $\|J_{f_\theta}(x)u\|_2^2$. Since the discrete regularizer aggregates neighboring states around each reference state, its small-neighborhood continuous limit induces an average over all unit directions around x .

To see how isotropic averaging yields Frobenius Jacobian energy, let u be uniformly distributed on the unit sphere $\mathbb{S}^{d-1}$. Then

$$\mathbb{E}_u [\|J_{f_\theta}(x)u\|_2^2] = \frac{1}{d} \|J_{f_\theta}(x)\|_F^2. \quad (4)$$

Thus, under a local isotropy assumption, the continuous counterpart of the discrete finite-difference regularizer is, up to a constant factor, a squared Frobenius Jacobian penalty. This establishes the theoretical bridge from discrete neighborhood smoothing to the continuous Jacobian regularization used by GPLD. In the next section, we instantiate this penalty in DreamerV3 by applying it row-wise to the posterior latent probability map. Proofs of (3) and (4) are deferred to Appendix A.

4 Gradient-Penalized Latent Dynamics in DreamerV3

We now instantiate the Frobenius Jacobian penalty motivated in Section 3 within DreamerV3. GPLD regularizes the posterior latent probability map rather than the raw environment transition map. This choice matches the object used for latent-state inference during world-model learning and yields a direct implementation through reverse-mode automatic differentiation.

DreamerV3 trains a recurrent state-space world model with deterministic latent state h_t , stochastic latent state z_t , encoder output e_t , and action a_t , all at time t . Its world-model objective can be written as

$$\mathcal{L}_{\text{Dv3}} := \mathbb{E}_{q_\phi} \left[\sum_{t=1}^T (\beta_{\text{pred}} \mathcal{L}_{\text{pred},t} + \beta_{\text{dyn}} \mathcal{L}_{\text{dyn},t} + \beta_{\text{rep}} \mathcal{L}_{\text{rep},t}) \right],$$

where $\mathcal{L}_{\text{pred},t}$ combines reconstruction, reward, and continuation prediction losses at time t . The dynamics and representation KL terms are

$$\mathcal{L}_{\text{dyn},t} = \max(1, D_{\text{KL}}(\text{sg}[q_\phi(z_t | h_t, e_t)] \| p_\phi(z_t | h_t))),$$

and

$$\mathcal{L}_{\text{rep},t} = \max(1, D_{\text{KL}}(q_\phi(z_t | h_t, e_t) \| \text{sg}[p_\phi(z_t | h_t)])),$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operator, which prevents gradients from flowing through its argument.

Let $u_t := [h_t, e_t]$ denote the posterior input. After row-wise softmax, the posterior distribution $q_\phi(z_t | h_t, e_t)$ is represented as a $K \times C$ probability table

$$q_\phi(u_t) \in [0, 1]^{K \times C},$$

where K is the number of categorical variables and C is the number of classes per variable. We refer to the map $u_t \mapsto q_\phi(u_t)$ as the posterior latent probability map. Let $q_\phi^{(i)}(u_t) \in \Delta^{C-1}$ denote the i -th row, where Δ^{C-1} is the probability simplex over C classes.

GPLD penalizes the average row-wise Frobenius Jacobian norm of this posterior latent probability map with respect to u_t :

$$\mathcal{R}_{\text{GPLD}}(u_t) = \frac{1}{K} \sum_{i=1}^K \|J_{q_\phi^{(i)}}(u_t)\|_F^2. \quad (5)$$

The resulting world-model objective is

$$\mathcal{L}_{\text{GPLD}} := \mathbb{E}_{q_\phi} \left[\sum_{t=1}^T (\beta_{\text{pred}} \mathcal{L}_{\text{pred},t} + \beta_{\text{dyn}} \mathcal{L}_{\text{dyn},t} + \beta_{\text{rep}} \mathcal{L}_{\text{rep},t} + \lambda_t^{\text{post}} \mathcal{R}_{\text{GPLD}}(u_t)) \right].$$

Computing the full Jacobian in (5) explicitly is expensive. We therefore estimate each row-wise Frobenius term using a Hutchinson-style output-space probe. For each posterior row i , sample $\epsilon_i \sim \text{Rad}^C$. Then

$$g_i(u_t) := \nabla_{u_t} \langle \epsilon_i, q_\phi^{(i)}(u_t) \rangle = J_{q_\phi^{(i)}}(u_t)^\top \epsilon_i.$$

Algorithm 1 Gradient penalty on the posterior latent probability map using Hutchinson’s estimator

```
1: Input: batch of latent states  $h$ , encoded observations  $e$ , posterior model  $q(h, e)$ , penalty coefficient  $\lambda_T$ , sampling fraction  $\rho$ 
2: Output: posterior gradient penalty  $\lambda_T L_{\text{gp}}$ 
3:  $B \leftarrow$  total batch size
4:  $N \leftarrow \lfloor \rho B \rfloor$  // number of sampled states
5: sample index set  $I \subset \{1, \dots, B\}$  with  $|I| = N$ 
6:  $\mathcal{L}_{\text{GPLD}} \leftarrow 0$ 
7: for each index  $k \in I$  do
8:   extract  $h_k, e_k$  from the batch
9:    $x_k \leftarrow [h_k, e_k]$ 
10:   $\mathbf{q}_k \leftarrow q(h_k, e_k) \in [0, 1]^{32 \times 16}$  // row-wise probabilities
11:  for  $i = 1$  to  $K$  do
12:    sample Rademacher noise  $\epsilon_{k,i} \sim \{-1, +1\}^C$ 
13:     $s_{k,i} \leftarrow \langle \epsilon_{k,i}, \mathbf{q}_{k,i} \rangle$ 
14:     $g_{k,i} \leftarrow \nabla_{x_k} s_{k,i}$ 
15:     $L_{\text{gp}} \leftarrow L_{\text{gp}} + \|g_{k,i}\|_2^2$ 
16:  end for
17: end for
18:  $L_{\text{gp}} \leftarrow \frac{1}{K \cdot N} L_{\text{gp}}$ 
19: return  $\lambda_T L_{\text{gp}}$ 
```

Since $\mathbb{E}[\epsilon_i \epsilon_i^\top] = I_C$, we have

$$\mathbb{E}_{\epsilon_i} \left[\left\| J_{q_\phi^{(i)}}(u_t)^\top \epsilon_i \right\|_2^2 \right] = \left\| J_{q_\phi^{(i)}}(u_t) \right\|_F^2.$$

Thus, the implemented estimator is

$$\hat{\mathcal{R}}_{\text{GPLD}}(u_t) = \frac{1}{K} \sum_{i=1}^K \left\| J_{q_\phi^{(i)}}(u_t)^\top \epsilon_i \right\|_2^2.$$

Algorithm 1 summarizes the minibatch implementation.

4.1 Why regularize only the posterior?

GPLD regularizes the posterior latent probability map rather than both the posterior and prior. This targets the observation-conditioned distribution $q_\phi(z_t | h_t, e_t)$, which is used to infer latent states from real trajectories during world-model training. Since the posterior depends directly on the encoded observation e_t , regularizing this map constrains the latent representation learned from environment data.

The prior $p_\phi(z_t | h_t)$ is not directly penalized by GPLD, but it is trained against the posterior through the dynamics KL term $\mathcal{L}_{\text{dyn},t}$. Thus, posterior-only regularization directly smooths the observation-conditioned latent map while still influencing the target that the prior learns to predict. Ablations in Section 5.4 show that this choice provides the best performance–cost trade-off among posterior-only, prior-only, and joint regularization.

4.2 Time-decayed gradient penalty coefficient

A fixed smoothness penalty can be useful early in training, when the world model is poorly estimated, but may become overly restrictive later as the model fit improves. We therefore decay the posterior penalty coefficient over training. Our default schedule uses square-root decay with a minimum threshold:

$$\lambda_T^{\text{post}} = \max \left(\frac{\lambda_0^{\text{post}}}{\sqrt{1 + T_{\text{updates}}/c}}, \lambda_{\min} \right),$$

where T_{updates} is the cumulative number of optimizer updates and $c > 0$ is a decay-scale constant. The square-root form is motivated by the finite-state intuition that explicit smoothing should matter

Algorithm 2 Square-root decay of the gradient penalty coefficient

- 1: **Input:** initial coefficient λ_0 , current update step T_{updates} , decay scale c , minimum coefficient $\lambda_{\min}$
 - 2: **Output:** decayed coefficient λ_T
 - 3: $s \leftarrow 1 + T_{\text{updates}}/c$
 - 4: $\lambda_T \leftarrow \max(\lambda_0/\sqrt{s}, \lambda_{\min})$
 - 5: **return** λ_T
-

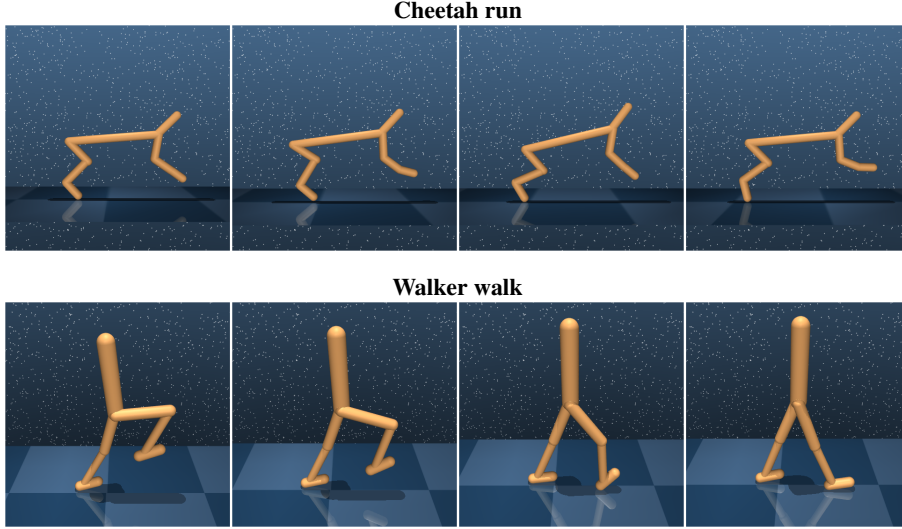


Figure 1: Representative frames from DMC locomotion tasks. Many proprioceptive locomotion environments evolve through smooth body motions over short time intervals, making them a natural testbed for evaluating local smoothness regularization in latent dynamics.

most when transition estimates are data-limited, and should weaken as more data are collected. Algorithm 2 summarizes the schedule.

5 Experiments

We evaluate GPLD on DeepMind Control (DMC) tasks (Tunyasuvunakool et al., 2020) to test whether local smoothness regularization improves latent world-model learning. Our main evaluation uses proprioceptive observations, where the agent receives low-dimensional physical state features. We additionally evaluate pixel observations, where the agent receives image frames and must learn visual representations jointly with latent dynamics.

5.1 Experimentation

DMC locomotion tasks provide a natural testbed for GPLD because many of them evolve through locally smooth body motions over short time intervals, as qualitatively illustrated in Fig. 1. We compare GPLD-DreamerV3 against DreamerV3 using aggregate learning curves, representative task-level curves, and ablations over the main design choices. Experimental details, including hyperparameters, seed counts, evaluation protocol, and aggregate normalization, are provided in Appendix B.

5.2 DMC Proprioceptive Tasks

GPLD provides the strongest evidence for its effectiveness in proprioceptive control, where the world model learns from low-dimensional physical state features and the local-smoothness prior acts directly on latent dynamics learning. Across the proprioceptive DMC benchmark, GPLD-DreamerV3 improves sample efficiency over DreamerV3, as shown in Fig. 2. Importantly, these aggregate gains

Table 1: Performance at 1M environment steps on higher-complexity proprioceptive DMC tasks. Individual task scores are raw episodic returns. For the aggregate summary reported in the text, scores are normalized by the final DreamerV3 baseline score for each task before aggregation.

Task	DreamerV3	GPLD-DreamerV3	Relative gain
cheetah-run	672.6	762.8	+13.4%
hopper-hop	81.8	197.4	+141.3%
hopper-stand	683.6	730.5	+6.9%
walker-run	458.5	627.1	+36.8%
walker-stand	942.8	957.8	+1.6%
walker-walk	878.3	946.2	+7.7%

are obtained using a single GPLD configuration across tasks, without per-environment tuning. The improvement appears in both raw aggregate return and normalized aggregate return, indicating that GPLD does not merely improve a single high-return environment, but produces broader gains across the benchmark.

The largest gains appear on tasks that demand coordinated locomotion and richer transition modeling. On the higher-complexity proprioceptive locomotion subset in Table 1, GPLD improves the normalized aggregate score by 34.6% at 1M environment steps. The improvement is especially large on hopper-hop and walker-run, while gains are smaller on tasks where DreamerV3 already approaches high return. The 1M-step comparison across all proprioceptive tasks is reported in Appendix C, Table 3, where GPLD improves the normalized aggregate mean by 17.7%. Individual 2M-step learning curves for the proprioceptive suite are shown in Appendix C, Fig. 6.

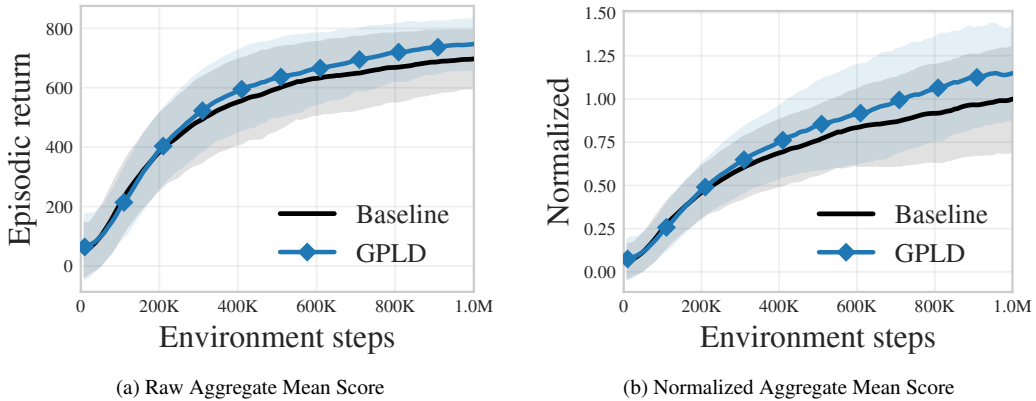


Figure 2: Aggregate proprioceptive-control performance on DMC tasks. GPLD-DreamerV3 improves sample efficiency over DreamerV3 in both raw aggregate return and normalized aggregate return.

Difficult quadruped tasks reveal a complementary long-horizon effect. Unlike the 1M-step locomotion subset, quadruped-run and quadruped-walk remain noisy early in training for both methods, suggesting that reliable latent dynamics for these environments require substantially more interaction. Over 4M environment steps, however, GPLD-DreamerV3 reaches the high-return regime earlier and maintains stronger late-stage performance than DreamerV3, as shown in Fig. 3. This indicates that the benefit of local smoothness regularization is not limited to early learning: on harder locomotion tasks, it can also improve the quality and consistency of learning over longer horizons.

5.3 DMC Pixel Tasks

We also evaluate GPLD with pixel observations, where the agent receives image frames rather than low-dimensional proprioceptive state features. This setting is harder because the world model must learn visual encodings and latent dynamics jointly. As shown in Fig. 4, GPLD-DreamerV3

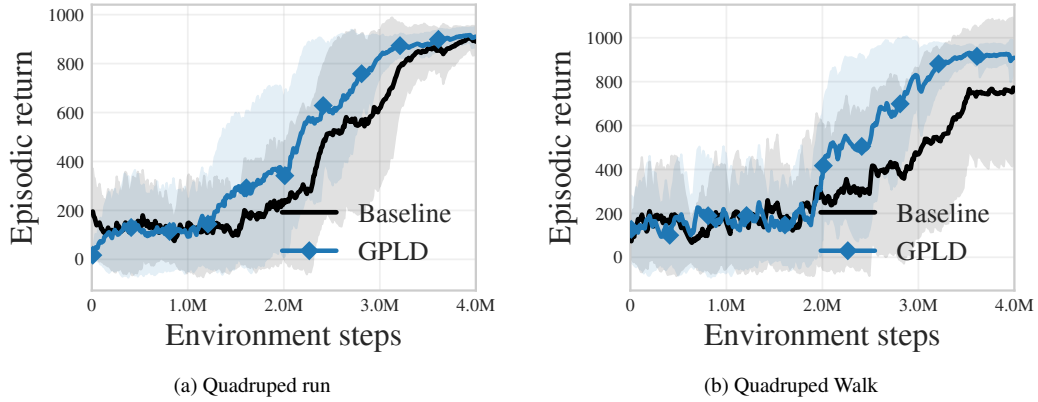


Figure 3: Long-horizon quadruped performance. Mean episodic return is reported across seeds, with shaded regions indicating one standard deviation around the mean. On quadruped-run and quadruped-walk, GPLD-DreamerV3 reaches high-return behavior earlier and maintains stronger late-stage performance than DreamerV3.

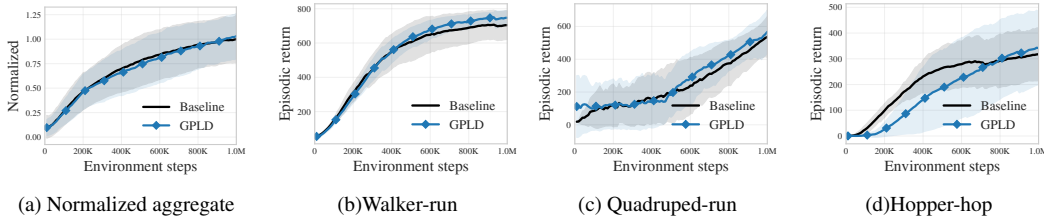


Figure 4: Pixel-observation DMC results. With image-frame observations, GPLD-DreamerV3 remains competitive in normalized aggregate performance but shows smaller gains than in proprioceptive control. Representative curves show clearer late-stage improvements on walker-run and quadruped-run, while hopper-hop exhibits slower early learning and a smaller late-stage advantage.

remains competitive with DreamerV3 in the normalized aggregate, but the gain is smaller than in proprioceptive control.

The task-level curves help explain this weaker aggregate effect. GPLD shows clearer late-stage gains on walker-run and quadruped-run, but hopper-hop is mixed: it learns more slowly early in training and catches up later. Thus, pixel results support a more cautious conclusion than the proprioceptive results. GPLD can still help under visual observations, but its effect is less direct when the same world model must simultaneously learn visual representations and smooth latent dynamics. Appendix D.1 provides individual pixel-observation learning curves for all evaluated tasks, while Appendix D.2 further studies an encoder-decoder warm-start diagnostic to separate the role of visual representation learning from latent-dynamics regularization.

5.4 Ablations

The ablations in Fig. 5 show that GPLD works best as a targeted posterior regularizer rather than as a broad smoothness constraint on all latent distributions. Posterior-only regularization performs competitively with or better than prior-only and joint prior-posterior regularization, while avoiding the extra derivative cost of explicitly penalizing both distributions. This supports the design choice in Section 4.1: GPLD directly regularizes the observation-conditioned latent map, while the prior remains coupled through the DreamerV3 dynamics loss.

The sampling fraction ρ controls the fraction of batch states on which the gradient penalty is evaluated, while the penalty coefficient controls the strength of the smoothness penalty on those states. Increasing ρ applies GPLD to more batch states, but the gains do not increase monotonically enough to justify

always using the largest value. The decay ablation further shows that maintaining a strong penalty throughout training is less effective than square-root decay, supporting the schedule in Section 4.2. Overall, the ablations indicate that GPLD is most effective when applied selectively to the posterior with moderate, time-decayed regularization.

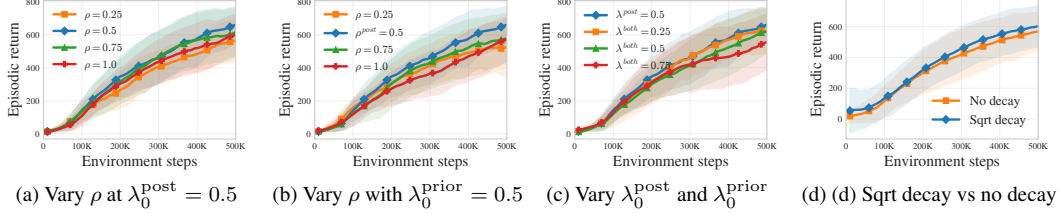


Figure 5: GPLD ablations over sampling fraction, posterior/prior regularization, and penalty scheduling. The results support using GPLD as a targeted posterior regularizer with time-decayed smoothness rather than a broad or fixed smoothness constraint.

5.5 Sensitivity diagnostics and computational cost

We include two additional diagnostics to check whether GPLD behaves as intended. To assess the effect of the gradient penalty, we measure how much the learned posterior and prior distributions change under small perturbations of their inputs. A smoother latent map should change less under such perturbations. GPLD reduces this measured local sensitivity on representative locomotion tasks, supporting the intended effect of the regularizer. Full details and plots are provided in Appendix F.

To assess computational overhead, we measure wall-clock runtime relative to DreamerV3. The additional cost comes from estimating the gradient penalty through vector-Jacobian products, and scales with the number of batch states selected by the sampling fraction ρ . Timing results are reported in Appendix E.

6 Conclusion

We have introduced GPLD, a local smoothness regularizer for latent world models, and instantiated it in DreamerV3 through a row-wise Jacobian penalty on the posterior latent probability map. This penalty is motivated by a discrete-to-continuous argument showing how finite-difference smoothing of neighboring transition laws leads to Frobenius Jacobian regularization in differentiable latent models. Empirically, GPLD improves aggregate sample efficiency on proprioceptive DMC tasks, with the strongest gains on higher-complexity locomotion environments. The quadrupled results further show that local smoothness regularization improves the quality and consistency of learning over longer horizons. These gains are obtained with a single GPLD configuration across tasks, without per-environment tuning.

There are some important limitations. GPLD is not uniformly beneficial across all environments, and its gains are weaker in pixel-observation settings, where latent dynamics learning is coupled to high-dimensional visual encoding. More broadly, GPLD encodes a local smoothness prior, so it may be less appropriate in environments where discontinuities dominate the relevant dynamics.

Impact Statement

This work aims to improve the sample efficiency of model-based reinforcement learning by regularizing learned latent dynamics. More sample-efficient world models can reduce the number of interactions needed to train continuous-control agents, potentially lowering simulation cost and making model-based RL more practical. As with other reinforcement-learning methods, downstream impact depends on the systems in which the trained agents are deployed. If such methods are later used in physical-control systems, deployment would require the same safety validation expected for other reinforcement-learning agents. Our experiments are limited to simulated DeepMind Control tasks and do not involve deployed robotic systems or human subjects.

Acknowledgements

Portions of this research were conducted with the advanced computing resources provided by Texas A&M High Performance Research Computing. This material is based upon work partially supported by the U.S. Army Contracting Command under Contract Numbers W911NF2120064, W911NF2520046, and W911NF2210151, the Office of Naval Research under Contracts N000142412615, and N00014-21-1-2385, and the National Science Foundation under Contract Numbers CNS-2328395 and CMMI-2038625. The views expressed herein and conclusions contained in this document are those of the authors and should not be interpreted as representing the views or official policies, either expressed or implied, of the U.S. Army, Navy, or the United States Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein.

References

- Kavosh Asadi, Dipendra Misra, and Michael Littman. Lipschitz continuity in model-based reinforcement learning. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 264–273. PMLR, 10–15 Jul 2018. URL <https://proceedings.mlr.press/v80/asadi18a.html>.
- Johan Bjorck, Carla P. Gomes, and Kilian Q. Weinberger. Towards deeper deep reinforcement learning with spectral normalization, 2022. URL <https://arxiv.org/abs/2106.01151>.
- Justin Boyan and Andrew Moore. Generalization in reinforcement learning: Safely approximating the value function. In G. Tesauro, D. Touretzky, and T. Leen, editors, *Advances in Neural Information Processing Systems*, volume 7. MIT Press, 1994. URL https://proceedings.neurips.cc/paper_files/paper/1994/file/ef50c335cca9f340bde656363ebd02fd-Paper.pdf.
- Zixuan Chen, Xialin He, Yen-Jen Wang, Qiayuan Liao, Yanjie Ze, Zhongyu Li, S. Shankar Sastry, Jiajun Wu, Koushil Sreenath, Saurabh Gupta, and Xue Bin Peng. Learning smooth humanoid locomotion through lipschitz-constrained policies, 2024. URL <https://arxiv.org/abs/2410.11825>.
- Guilherme Christmann, Ying-Sheng Luo, Hanjaya Mandala, and Wei-Chao Chen. Benchmarking smoothness and reducing high-frequency oscillations in continuous control policies, 2024. URL <https://arxiv.org/abs/2410.16632>.
- Benoit Dherin, Michael Munn, Mihaela Rosca, and David G. T. Barrett. Why neural networks find simple solutions: the many regularizers of geometric complexity, 2022. URL <https://arxiv.org/abs/2209.13083>.
- Zipeng Fu, Ashish Kumar, Jitendra Malik, and Deepak Pathak. Minimizing energy consumption leads to the emergence of gaits in legged robots, 2021. URL <https://arxiv.org/abs/2111.01674>.
- Florin Gogianu, Tudor Berariu, Mihaela Rosca, Claudia Clopath, Lucian Busoniu, and Razvan Pascanu. Spectral normalisation for deep reinforcement learning: an optimisation perspective, 2021. URL <https://arxiv.org/abs/2105.05246>.
- Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples, 2015. URL <https://arxiv.org/abs/1412.6572>.
- Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein gans, 2017. URL <https://arxiv.org/abs/1704.00028>.
- David Ha and Jürgen Schmidhuber. World models, 2018. URL <https://zenodo.org/record/1207631>.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025. doi: 10.1038/s41586-025-08744-2.

- Rémi Munos and Csaba Szepesvári. Finite-time bounds for fitted value iteration. *Journal of Machine Learning Research*, 9(27):815–857, 2008. URL <https://www.jmlr.org/papers/v9/munos08a.html>.
- Xue Bin Peng, Erwin Coumans, Tingnan Zhang, Tsang-Wei Lee, Jie Tan, and Sergey Levine. Learning agile robotic locomotion skills by imitating animals, 2020. URL <https://arxiv.org/abs/2004.00784>.
- Mihaela Rosca, Theophane Weber, Arthur Gretton, and Shakir Mohamed. A case for new neural network smoothness constraints. In Jessica Zosa Forde, Francisco Ruiz, Melanie F. Pradier, and Aaron Schein, editors, *Proceedings on "I Can't Believe It's Not Better!" at NeurIPS Workshops*, volume 137 of *Proceedings of Machine Learning Research*, pages 21–32. PMLR, 12 Dec 2020. URL <https://proceedings.mlr.press/v137/rosca20a.html>.
- Dávid Terjék. Adversarial lipschitz regularization, 2020. URL <https://arxiv.org/abs/1907.05681>.
- Sebastian Thrun and Anton Schwartz. Issues in using function approximation for reinforcement learning. In Michael Mozer, Paul Smolensky, David Touretzky, Jeffrey Elman, and Andreas Weigend, editors, *Proceedings of the 1993 Connectionist Models Summer School*, pages 255–263. Lawrence Erlbaum, 1993. URL http://www.ri.cmu.edu/pub_files/pub1/thrun_sebastian_1993_1/thrun_sebastian_1993_1.pdf.
- Saran Tunyasuvunakool, Alistair Muldal, Yotam Doron, Siqi Liu, Steven Bohez, Josh Merel, Tom Erez, Timothy Lillicrap, Nicolas Heess, and Yuval Tassa. dm-control: Software and tasks for continuous control. *Software Impacts*, 6:100022, 2020. ISSN 2665-9638. doi: <https://doi.org/10.1016/j.simpa.2020.100022>. URL <https://www.sciencedirect.com/science/article/pii/S2665963820300099>.
- Arun Venkatraman, Martial Hebert, and J. Bagnell. Improving multi-step prediction of learned time series models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 29, 02 2015. doi: 10.1609/aaai.v29i1.9590.
- Haoran Wang, Yaoru Sun, Zeshen Tang, Haibo Shi, and Chenyuan Jiao. Hg2p: Hippocampus-inspired high-reward graph and model-free q-gradient penalty for path planning and motion control. *Neural Networks*, 191:107781, 2025. ISSN 0893-6080. doi: <https://doi.org/10.1016/j.neunet.2025.107781>. URL <https://www.sciencedirect.com/science/article/pii/S0893608025006616>.

A Proofs for the discrete-to-continuous justification of GPLD

1. Proof of Finite Differences converging to directional Jacobian sensitivities (3)

Proof. Since $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^m$ is locally Fréchet differentiable at x , there exists a linear map $J_{f_\theta}(x) \in \mathbb{R}^{m \times d}$ such that

$$f_\theta(x + hu) = f_\theta(x) + J_{f_\theta}(x)(hu) + r(h), \quad (6)$$

where the remainder satisfies

$$\frac{\|r(h)\|_2}{|h|} \rightarrow 0 \quad \text{as } h \rightarrow 0. \quad (7)$$

Dividing (6) by h gives

$$\frac{f_\theta(x + hu) - f_\theta(x)}{h} = J_{f_\theta}(x)u + \frac{r(h)}{h}. \quad (8)$$

By (7), we have

$$\left\| \frac{r(h)}{h} \right\|_2 \rightarrow 0 \quad \text{as } h \rightarrow 0.$$

Therefore,

$$\frac{f_\theta(x + hu) - f_\theta(x)}{h} \rightarrow J_{f_\theta}(x)u \quad \text{as } h \rightarrow 0.$$

Taking squared ℓ_2 -norms on both sides and using continuity of the norm yields

$$\lim_{h \rightarrow 0} \left\| \frac{f_\theta(x + hu) - f_\theta(x)}{h} \right\|_2^2 = \|J_{f_\theta}(x)u\|_2^2.$$

This proves the claim. $\square$

2. Isotropic averaging of Directional Jacobian norm yields Frobenius Jacobian energy (4)

Proof. Let $J := J_{f_\theta}(x) \in \mathbb{R}^{m \times d}$. Then

$$\|Ju\|_2^2 = u^\top J^\top Ju. \quad (9)$$

Taking expectation over $u \sim \text{Unif}(\mathbb{S}^{d-1})$, we obtain

$$\mathbb{E}_u[\|Ju\|_2^2] = \mathbb{E}_u[u^\top J^\top Ju]. \quad (10)$$

Using the trace identity $v^\top Av = \text{Tr}(Avv^\top)$, we have

$$\mathbb{E}_u[u^\top J^\top Ju] = \mathbb{E}_u[\text{Tr}(J^\top J uu^\top)] = \text{Tr}(J^\top J \mathbb{E}_u[uu^\top]). \quad (11)$$

Since u is uniformly distributed on the unit sphere, its distribution is isotropic, and hence

$$\mathbb{E}_u[uu^\top] = \frac{1}{d}I_d. \quad (12)$$

Substituting (12) into (11) yields

$$\mathbb{E}_u[\|Ju\|_2^2] = \text{Tr}\left(J^\top J \cdot \frac{1}{d}I_d\right) = \frac{1}{d} \text{Tr}(J^\top J).$$

Finally, by the definition of the Frobenius norm,

$$\text{Tr}(J^\top J) = \|J\|_F^2.$$

Therefore,

$$\mathbb{E}_u[\|J_{f_\theta}(x)u\|_2^2] = \frac{1}{d} \|J_{f_\theta}(x)\|_F^2,$$

as claimed. $\square$

B Experimental Details

We use the open-source DreamerV3 implementation, which is released under the MIT license, and the DeepMind Control Suite, which is released under the Apache-2.0 license.

Unless otherwise stated, GPLD uses posterior regularization with $\lambda_0^{\text{post}} = 0.5$, no prior regularization, the square-root decay schedule from Algorithm 2, imagination horizon $H = 25$, sampling fraction $\rho = 0.5$, decay scale $c = 1000$, and minimum coefficient $\lambda_{\min} = 0.001$. We use the 12M-parameter DreamerV3 configuration and keep all non-GPLD hyperparameters fixed between the baseline and GPLD variants.

Our primary metric is sample efficiency, measured by evaluation return as a function of environment steps. Learning curves report mean episodic return across available seeds, with shaded regions indicating one standard deviation around the mean; unless otherwise noted, experiments use five seeds. For aggregate curves, we report both raw and normalized aggregate return. In the normalized aggregate, each task is scaled by its final DreamerV3 baseline score before averaging, so that tasks contribute equally relative to their baseline performance.

Table 2 lists the main training and evaluation hyperparameters used in our experiments. DreamerV3 and GPLD-DreamerV3 share all non-GPLD hyperparameters; the only differences are the posterior gradient penalty, sampling fraction, and decay schedule.

C DMC Proprioceptive Results

Fig 6 shows that the early GPLD gains generally persist or remain competitive over longer horizons, while quadruped tasks are separated in the main text because their benefits emerge most clearly only beyond the 1M-step regime.

Table 2: Training and evaluation hyperparameters. All non-GPLD hyperparameters are shared between DreamerV3 and GPLD-DreamerV3. Values are taken from the 12M DMC configuration, with GPLD-specific overrides listed separately.

Hyperparameter	DreamerV3	GPLD-DreamerV3
World-model size	12M	12M
Benchmark	DeepMind Control	DeepMind Control
Environment action repeat	2	2
Image resolution	64×64	64×64
Number of parallel environments	16	16
Training steps	10^6	10^6
Train ratio, proprioceptive	512	512
Train ratio, pixel	512	512
Batch size	16	16
Batch length	64	64
Replay capacity	5×10^6	5×10^6
Evaluation frequency	5000 env. steps	5000 env. steps
Evaluation episodes during training	3	3
Optimizer learning rate	4×10^{-5}	4×10^{-5}
Optimizer β_1, β_2	0.9, 0.999	0.9, 0.999
Optimizer epsilon	10^{-20}	10^{-20}
Weight decay	0	0
AGC coefficient	0.3	0.3
RSSM deterministic size	2048	2048
RSSM stochastic variables	32	32
RSSM classes per variable	16	16
RSSM hidden size	256	256
Network depth	16	16
Network units	256	256
Loss scales ($\beta_{\text{pred}}, \beta_{\text{dyn}}, \beta_{\text{rep}}$)	(1.0, 1.0, 0.1)	(1.0, 1.0, 0.1)
Imagination horizon	25	25
λ_0^{post}	0	0.5
λ_0^{prior}	0	0
Sampling fraction ρ	—	0.5
Penalty decay	—	square-root decay
Decay scale c	—	1000
Minimum coefficient λ_{min}	—	0.001

D DMC Pixel Results

D.1 Individual learning plots

Fig. 7 shows individual plots for the DMC Vision tasks.

D.2 Encoder-decoder warm-start diagnostic for pixel observations

To better understand why GPLD yields weaker aggregate gains under pixel observations, we run an encoder-decoder warm-start diagnostic. In this setting, the encoder and decoder are initialized from a previously trained DreamerV3 baseline run, while all other components are reinitialized. The encoder and decoder are not frozen; they continue to train during the new run. Both DreamerV3 and GPLD-DreamerV3 use the same encoder-decoder initialization, so differences in performance reflect how the methods use the improved visual initialization rather than a different starting representation.

Figure 8 shows results on hopper-hop and quadruped-walk. Under the same warm start, GPLD improves performance over DreamerV3 on these tasks. This supports the interpretation that the weaker fully end-to-end pixel gains may partly arise because visual representation learning and latent-dynamics learning must be solved simultaneously. We treat this experiment as a diagnostic rather than a main empirical claim.

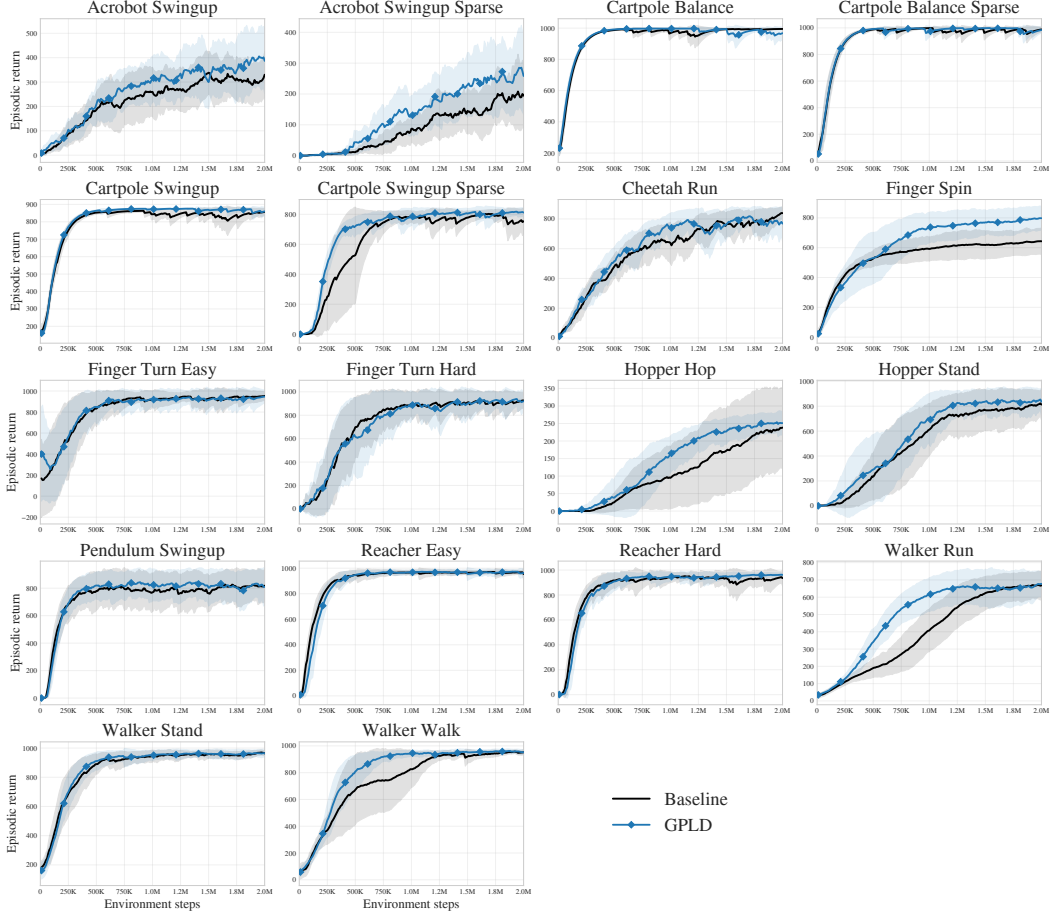


Figure 6: DMC Proprioceptive Individual Tasks

E Computational Cost Analysis

Computation cost analysis: (Avg Baseline time = 2:01:29/seed for 500k env steps.)

GPLD introduces additional computation because the gradient penalty requires vector-Jacobian products. The main algorithmic cost driver is the sampling fraction ρ , which determines the fraction of batch states on which the penalty is evaluated. In contrast, changing the penalty coefficient λ^{post} does not substantially change the number of derivative evaluations and should not, by itself, determine the algorithmic overhead.

Table 4 reports wall-clock overhead relative to vanilla DreamerV3. For the default setting used in our main experiments, $\lambda_0^{\text{post}} = 0.5$ and $\rho = 0.5$, the measured runtime ratio is 1.51. However, this value should be interpreted cautiously: the standard deviation is relatively large, and a nearby posterior-only run with the same sampling fraction, $\lambda_0^{\text{post}} = 0.25$ and $\rho = 0.5$, has a substantially lower runtime ratio of 1.23. Since these two settings require the same type of gradient-penalty computation, the difference suggests that the 1.51 measurement likely reflects wall-clock variability in the execution environment in addition to the algorithmic overhead of GPLD.

Overall, the timing results indicate that GPLD adds measurable overhead relative to DreamerV3, but the $1.51\times$ value should be viewed as a conservative high-end wall-clock estimate rather than the inherent cost of the method. Runs with the same sampling fraction suggest a more typical overhead closer to the $1.2\text{--}1.3\times$ range, although precise runtime depends on hardware utilization and system-level variability.

Table 3: Full 1M-step proprioceptive DMC results. Scores are reported as mean $\pm$ standard deviation across seeds, using checkpoint evaluation with 20 episodes per seed. GPLD-DreamerV3 improves the normalized aggregate mean by 17.7%.

Task	DreamerV3	GPLD-DreamerV3	Relative gain
acrobot-swingup	238.1 $\pm$ 15.5	304.6 $\pm$ 15.4	+27.9%
acrobot-swingup-sparse	99.7 $\pm$ 18.0	135.6 $\pm$ 58.4	+36.0%
cartpole-balance	990.0 $\pm$ 6.6	997.1 $\pm$ 0.0	+0.7%
cartpole-balance-sparse	999.9 $\pm$ 0.1	1000.0 $\pm$ 0.0	+0.0%
cartpole-swingup	860.3 $\pm$ 0.6	870.3 $\pm$ 4.1	+1.2%
cartpole-swingup-sparse	729.3 $\pm$ 57.1	816.2 $\pm$ 29.2	+11.9%
cheetah-run	672.6 $\pm$ 15.3	762.8 $\pm$ 39.0	+13.4%
finger-spin	598.2 $\pm$ 79.4	735.2 $\pm$ 97.3	+22.9%
finger-turn-easy	941.4 $\pm$ 8.6	903.2 $\pm$ 39.6	-4.1%
finger-turn-hard	853.0 $\pm$ 75.8	882.5 $\pm$ 55.1	+3.5%
hopper-hop	81.8 $\pm$ 79.0	197.4 $\pm$ 41.7	+141.4%
hopper-stand	683.6 $\pm$ 156.0	730.5 $\pm$ 125.0	+6.9%
pendulum-swingup	760.7 $\pm$ 121.3	821.2 $\pm$ 18.4	+8.0%
reacher-easy	947.1 $\pm$ 28.5	968.1 $\pm$ 8.6	+2.2%
reacher-hard	943.9 $\pm$ 30.5	947.9 $\pm$ 23.9	+0.4%
walker-run	458.5 $\pm$ 122.8	627.1 $\pm$ 77.2	+36.8%
walker-stand	942.8 $\pm$ 16.1	957.8 $\pm$ 8.5	+1.6%
walker-walk	878.3 $\pm$ 86.9	946.2 $\pm$ 12.7	+7.7%

Table 4: Computation-time overhead of GPLD relative to vanilla DreamerV3 across different $(\rho, \lambda^{post}, \lambda^{prior})$ settings. Runtime is reported as the mean ratio over runs, with standard deviation.

ρ	λ^{post}	λ^{prior}	Mean $\pm$ Std (ratio)	Diff (s)
0.25	0	0.5	1.01 $\pm$ 0.022	74.40
0.25	0.5	0	1.11 $\pm$ 0.063	770.10
0.5	0	0.5	1.15 $\pm$ 0.032	1108.00
0.5	0.25	0	1.23 $\pm$ 0.028	1668.00
0.5	0.25	0.25	1.44 $\pm$ 0.027	3200.60
0.5	0.5	0	1.51 $\pm$ 0.157	3712.10
0.5	0.5	0.5	1.44 $\pm$ 0.027	3177.40
0.5	0.75	0.75	1.43 $\pm$ 0.037	3118.40
0.75	0	0.5	1.26 $\pm$ 0.026	1927.30
0.75	0.5	0	1.46 $\pm$ 0.059	3381.80
1.0	0	0.5	1.40 $\pm$ 0.029	2895.20
1.0	0.5	0	1.68 $\pm$ 0.100	4920.40

F Local sensitivity analysis

To verify that GPLD changes the learned world model in the intended direction, we estimate the local sensitivity of the posterior and prior distributions during training. For each saved checkpoint, we generate trajectories and perturb the corresponding latent inputs. For the posterior, we perturb the encoder-dependent input e_t ; for the prior, we perturb the deterministic state h_t . We then measure the change in the output distribution using KL divergence and normalize by the perturbation norm.

Let $r_\phi(\cdot \mid u_t)$ denote either the posterior or prior distribution evaluated at its corresponding input u_t . For a perturbation δ , we estimate

$$\frac{D_{\text{KL}}(r_\phi(\cdot \mid u_t) \parallel r_\phi(\cdot \mid u_t + \delta))}{\|\delta\|_2},$$

with perturbation magnitudes ranging from 0.1% to 10% of the average input norm. We average this quantity across time steps, perturbation magnitudes, and seeds. Lower values indicate that the learned latent distribution changes less under small input perturbations.

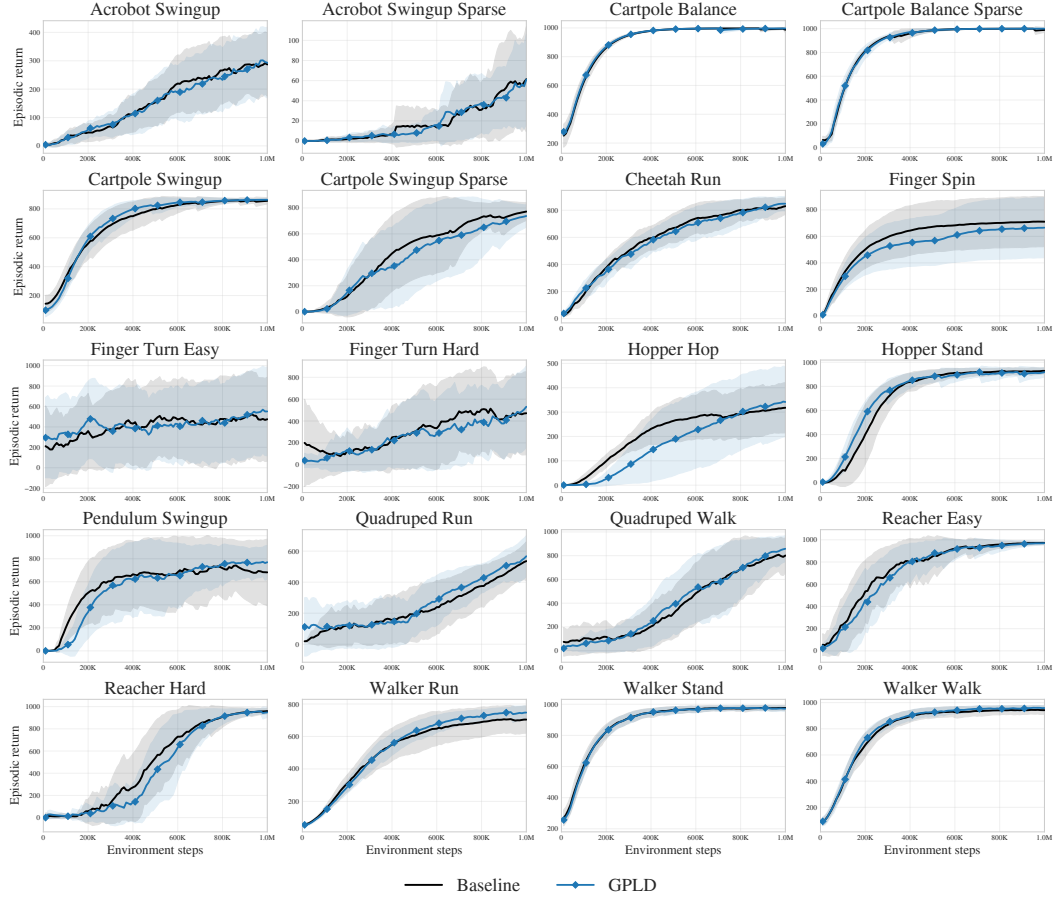


Figure 7: DMC Pixel Individual Tasks

Figure 9 shows aggregate local sensitivity across tasks. GPLD reduces the sensitivity of the learned posterior distribution relative to DreamerV3, consistent with the intended effect of the gradient penalty. The prior sensitivity is also reduced in several cases, even though GPLD is applied directly only to the posterior, suggesting that the posterior regularizer influences the prior through the DreamerV3 dynamics loss.

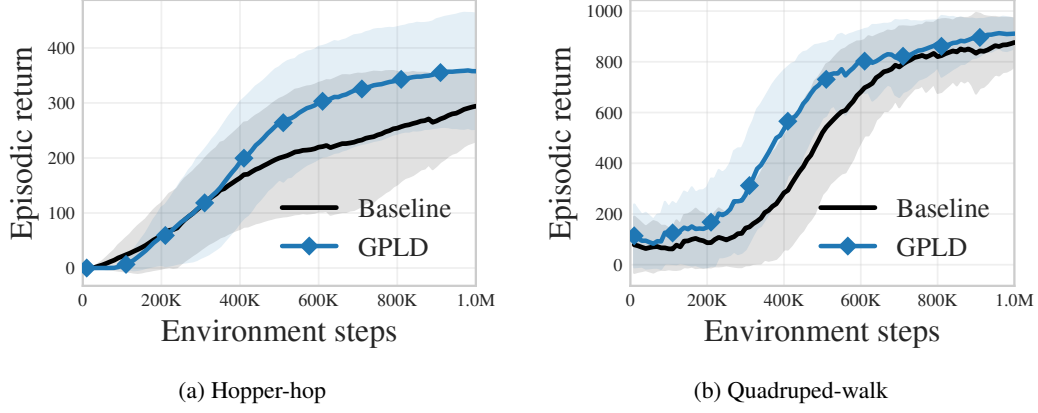


Figure 8: Encoder-decoder warm-start diagnostic for pixel observations. The encoder and decoder are initialized from a trained DreamerV3 baseline run and continue training, while all other components are reinitialized. With the same visual warm start, GPLD-DreamerV3 improves performance on hopper-hop and quadraped-walk, suggesting that visual representation learning may partially mask the effect of latent-dynamics regularization in fully end-to-end pixel training.

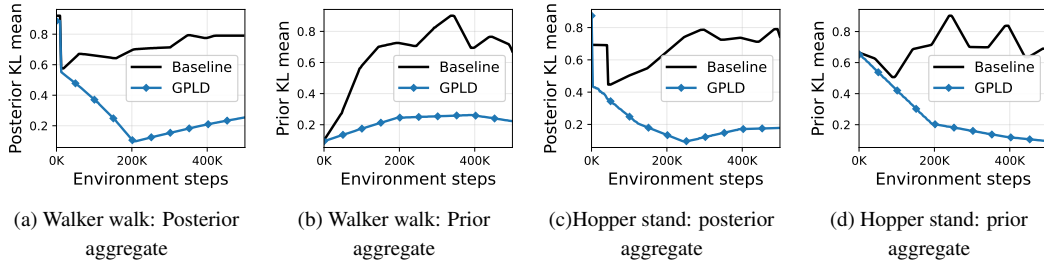


Figure 9: Aggregate local sensitivity of the learned posterior and prior distributions for Walker walk and Hopper stand environments. We perturb the corresponding latent inputs and measure the KL change in the output distribution normalized by the perturbation norm. GPLD reduces posterior sensitivity relative to DreamerV3, consistent with the intended effect of the gradient penalty.